

05

/ 13 LECTURES

Graph Algorithms for Networking

Networks are graphs. Learn BFS, DFS and Dijkstra on a real network — and see exactly which routing protocol is powered by each algorithm.

Graph Representation

Adjacency List · Matrix

BFS — Queue

DFS — Stack

Cycle Detection

Dijkstra

Shortest Path

RIP · OSPF mapping

WHAT THIS LECTURE GIVES YOU

A complete, traced tour of the three algorithms behind Internet routing: representing a network as a weighted graph, BFS for fewest hops (→ RIP), DFS for topology exploration and loop detection, and Dijkstra for least-cost paths (→ OSPF) — with Python code, step-by-step traces, and a full Dijkstra table you can reproduce in the exam.

STUDY-READY EDITION

CONCEPTS · CODE · TRACES · EXAM PREP

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

- Model a network as a **weighted graph** and choose the right representation
- Implement and **trace BFS** for hop-count shortest paths
- Implement and **trace DFS** for topology exploration and cycle detection
- Apply **Dijkstra** for least-cost routing and reconstruct the path

Graphs for Networking



1 Networks as Graphs

A graph $G = (V, E)$: **V** = routers (vertices/nodes), **E** = links (edges), and each edge carries a **weight** = link cost (latency, bandwidth-derived cost, or hop cost). Our running example — a 5-city network:

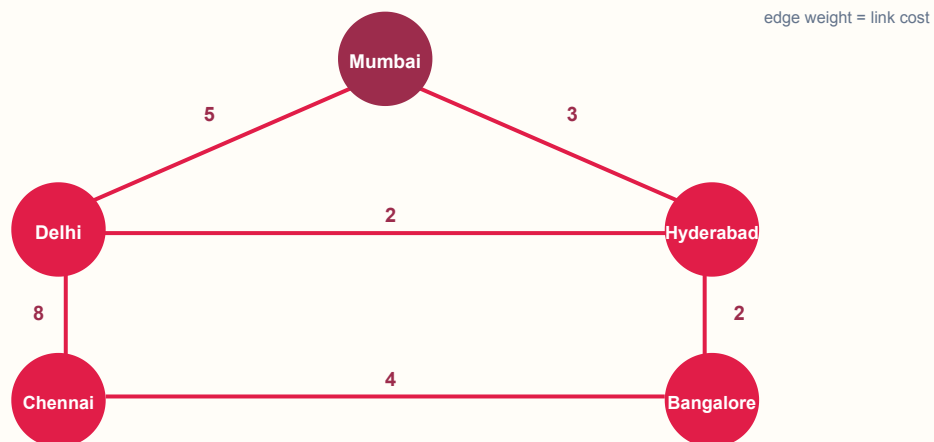


Figure 1. The running example graph used for every trace in this lecture.

ADJACENCY LIST – SPARSE GRAPHS ✓ PYTHON

```
graph = {
    'Mumbai': [('Delhi', 5), ('Hyderabad', 3)],
    'Delhi':   [('Mumbai', 5), ('Hyderabad', 2),
                ('Chennai', 8)],
    'Hyderabad': [('Mumbai', 3), ('Delhi', 2),
                  ('Bangalore', 2)],
    'Chennai':  [('Delhi', 8), ('Bangalore', 4)],
    'Bangalore': [('Hyderabad', 2), ('Chennai', 4)]
}
```

ADJACENCY MATRIX – DENSE GRAPHS PYTHON

#	M	D	H	C	B	
M	[0, 5, 3, ∞, ∞]	# Mumbai				
D	[5, 0, 2, 8, ∞]	# Delhi				
H	[3, 2, 0, ∞, 2]	# Hyd				
C	[∞, 8, ∞, 0, 4]	# Chennai				
B	[∞, ∞, 2, 4, 0]	# Bang				

CHOICE The Internet is **sparse** (each router connects to a few others) → adjacency **list** wins: $O(V+E)$ space vs $O(V^2)$ for the matrix.

2 BFS — Breadth-First Search

What BFS finds

The **minimum hop-count** path from source to every reachable node. Uses a **queue (FIFO)**; explores layer by layer: 1 hop away, then 2 hops, then 3...

Limitation

BFS minimises **hops, not cost**. A 1-hop path costing 100 beats a 3-hop path costing 3 — wrong for cost-based routing. For cost, use **Dijkstra** (§4).

BFS WITH PATH TRACKING

PYTHON

```
from collections import deque

def bfs_path(graph, start, end):
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        # FIFO — the queue is the whole trick
        if node == end:
            return path
            # first time we see end = fewest hops
        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None
```

Trace — BFS from Mumbai to Bangalore

Step	Visit	Queue after	Notes
init	—	[Mumbai]	visited = {Mumbai}
1	Mumbai	[Delhi, Hyderabad]	adds both neighbours
2	Delhi	[Hyderabad, Chennai]	Hyderabad already visited
3	Hyderabad	[Chennai, Bangalore]	adds Bangalore
4	Chennai	[Bangalore]	Bangalore already queued
5	Bangalore	[]	FOUND! Path: Mumbai → Hyderabad → Bangalore (2 hops)

3 DFS — Depth-First Search

DFS explores **as deep as possible** along one branch before backtracking — a **stack** (or recursion) instead of a queue. Finds a path, not necessarily the shortest. $O(V+E)$.

DFS — ITERATIVE (STACK) AND RECURSIVE

PYTHON

```
def dfs_iterative(graph, start):
    stack, visited, order = [start], set(), []
    while stack:
        node = stack.pop()          # LIFO — go deep first
        if node not in visited:
            visited.add(node); order.append(node)
            for neighbor, w in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)
    return order

def has_cycle(graph, node, visited, rec_stack):
    visited.add(node); rec_stack.add(node)
    for neighbor, _ in graph[node]:
        if neighbor not in visited:
            if has_cycle(graph, neighbor, visited, rec_stack): return True
        elif neighbor in rec_stack:
            return True          # back-edge = cycle!
    rec_stack.discard(node)
    return False
```

Trace — DFS from Mumbai

```
Visit Mumbai → push Delhi, Hyderabad
Pop Hyderabad → push Bangalore
Pop Bangalore → push Chennai
Pop Chennai → Delhi already seen
Pop Delhi → all neighbours visited
Order: Mumbai → Hyderabad → Bangalore →
Chennai → Delhi
```

Network uses of DFS

- **Topology discovery** — walk every reachable router
- **Cycle detection** — a cycle in the routing graph = a potential **routing loop** (packets circling forever)
- **Spanning-tree construction** (preview of Lecture 6 / STP)

4 Dijkstra — Minimum-Cost Paths

Idea

Greedy: always expand the cheapest unfinished node; relax its neighbours. A **min-heap** serves the cheapest node fast.

Guarantee

Optimal costs to ALL nodes — but only when **all edge weights ≥ 0** (why: Lecture 6).

Complexity

$O((V+E) \log V)$ with a binary heap — the price of the priority queue, worth it for correct costs.

DIJKSTRA + PATH RECONSTRUCTION

PYTHON

```
import heapq

def dijkstra(graph, source):
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    prev = {node: None for node in graph}
    pq = [(0, source)]  # (cost, node) min-heap
    while pq:
        cost, node = heapq.heappop(pq)
        if cost > dist[node]:  # stale entry — skip
            continue
        for neighbor, weight in graph[node]:
            new_cost = cost + weight
            if new_cost < dist[neighbor]:  # relax
                dist[neighbor] = new_cost
                prev[neighbor] = node
                heapq.heappush(pq, (new_cost, neighbor))
    return dist, prev

def reconstruct_path(prev, target):
    path, node = [], target
    while node is not None:
        path.append(node); node = prev[node]
    return list(reversed(path))
```

Full trace — Dijkstra from Mumbai

Step	Expanded	Delhi	Hyderabad	Chennai	Bangalore	What happened
init	—	∞	∞	∞	∞	dist[Mumbai]=0
1	Mumbai (0)	5	3	∞	∞	relax both links
2	Hyderabad (3)	5	3	∞	5	$3+2=5 < \infty \rightarrow$ improve B
3	Delhi (5)	5	3	13	5	$5+8=13 \rightarrow$ improve C
4	Bangalore (5)	5	3	9	5	$5+4=9 < 13 \rightarrow$ improve C
5	Chennai (9)	5	3	9	5	nothing improves — done

Final shortest paths from Mumbai:

→ Delhi: 5 (direct)
→ Hyderabad: 3 (direct)
→ Bangalore: 5 (Mumbai → Hyderabad → Bangalore)
→ Chennai: 9 (Mumbai → Hyderabad → Bangalore → Chennai)

NUMERICAL Reproduce this exact table in the exam: one row per expansion, always expand the current minimum, update only when strictly smaller. Chennai improved twice (13 → 9) — that's Dijkstra working.

5 BFS vs DFS vs Dijkstra

Feature	BFS	DFS	Dijkstra
Data structure	Queue (FIFO)	Stack / recursion	Min-heap
What it finds	Fewest hops	A path (any)	Minimum cost
Time complexity	$O(V + E)$	$O(V + E)$	$O((V+E) \log V)$
Uses weights?	No — ignores	No — ignores	Yes — required
Negative weights?	N/A	N/A	No — breaks (L6)
Network use	RIP (hop count)	Topology / loop detection	OSPF (link cost)

Algorithms → Real Protocols

Algorithm	Real protocol	Role
BFS (hop count)	RIP — Routing Information Protocol	metric = hop count
Dijkstra	OSPF — Open Shortest Path First	every router runs Dijkstra on the full topology
Bellman-Ford	RIP, BGP (conceptually)	distributed updates — next lecture
MST algorithms	STP — Spanning Tree Protocol	prevents Layer-2 loops

LECTURE 5 — RAPID REVISION CARD

Network = graph: routers V , links E , weight = cost.

Adjacency list for sparse real networks ($O(V+E)$);
matrix for dense/fast-lookup ($O(V^2)$).

BFS: queue → fewest hops → **RIP**. Explores in layers;
first arrival = shortest hop path.

DFS: stack → deep-first; cycle detection (back-edge) =
routing-loop finder; $O(V+E)$.

Dijkstra: min-heap, greedy, relax = "new cost < known?"; $O((V+E) \log V)$; **no negative weights**.

Mumbai example final costs: Delhi 5 · Hyd 3 · Bang 5 ·
Chennai 9.

Stale-heap-entry check: if `cost > dist[node]`:
continue.

Path reconstruction: follow `prev[]` from target back to
source, then reverse.



Important Questions — Exam Practice

A. Multiple Choice

Q1. Which data structure distinguishes BFS from DFS?

MCQ

- a) Heap vs array
- b) Queue vs stack
- c) Set vs list
- d) Graph vs tree

Q2. Dijkstra fails on negative edge weights because it:

MCQ

- a) uses too much memory
- b) finalises a node before a cheaper path through a negative edge appears
- c) cannot use a heap
- d) only works on trees

Q3. In the Figure-1 network, the minimum-cost path Delhi → Bangalore is:

MCQ

- a) Delhi → Hyderabad → Bangalore = 4
- b) Delhi → Mumbai → Hyderabad → Bangalore = 10
- c) Delhi → Chennai → Bangalore = 12
- d) direct — no such link

Q4. Why is an adjacency list preferred for Internet-scale routing?

MCQ

- a) It sorts edges automatically
- b) The Internet is sparse — $O(V+E)$ storage beats $O(V^2)$
- c) Matrices cannot store weights
- d) Lists enable BFS only

Q5. Which real protocol runs Dijkstra on a full topology map?

MCQ

- a) RIP
- b) OSPF
- c) ARP
- d) DHCP

B. Short Answer (2–4 marks)

Q6. Model a network as a weighted graph: what are vertices, edges, weights? Give two realistic weight choices.

SHORT

Q7. When would you choose an adjacency matrix over a list? What is the space complexity of each?

SHORT

Q8. Why can't BFS give minimum-cost paths? Give the concrete 1-hop vs 3-hop example.

SHORT

Q9. How does DFS detect a cycle? What does a cycle mean in a routing graph?

SHORT

Q10. Why does Dijkstra's code check `if cost > dist[node]: continue`?

SHORT

C. Long Answer / Numericals (5–8 marks)

Q11. Trace BFS from **Chennai** on Figure 1: give the visit order and the hop-distance of every node.

NUMERICAL

Q12. Run Dijkstra from **Delhi** on Figure 1. Show the table (one row per expansion), final costs to all nodes, and the shortest path Delhi → Chennai.

NUMERICAL

Q13. Write Dijkstra's pseudocode and explain the role of the min-heap, the relaxation step, and the prev[] map. State time complexity.

LONG

Q14. "Routing protocols are graph algorithms in disguise." Justify with the BFS↔RIP and Dijkstra↔OSPF mappings, and explain what a routing loop is using DFS cycle language.

LONG

ANSWER KEY

A1. b — queue (FIFO) vs stack (LIFO).

A2. b — greedy finalisation is wrong when a longer route later becomes cheaper.

A3. a — $2 + 2 = 4$ beats 10 and 12.

A4. b — sparse → list is compact and fast.

A5. b — OSPF; RIP is BFS-like hop count.

A6. V = routers, E = links, weight = cost. Realistic: latency (ms) or bandwidth-derived cost (Lecture 8).

A7. Dense graphs / constant-time edge lookup. List $O(V+E)$; matrix $O(V^2)$.

A8. BFS ignores weights — 1 hop costing 100 "wins" over 3 hops costing 3.

A9. A back-edge to a node in the current recursion stack. In routing = packet-loop risk.

A10. Lazy deletion: a better route may have popped first; stale heap entries must be skipped.

A11. Order: Chennai → Delhi → Bangalore → Mumbai → Hyderabad. Hops: Delhi 1, Bangalore 1, Mumbai 2, Hyderabad 2.

A12. Expansions: Delhi(0) → Mumbai(5), Hyderabad(2) → Bangalore(4) → Chennai(8)... full: Delhi 0, Hyderabad 2, Mumbai 5, Bangalore 4, Chennai 8; Delhi→Chennai = Delhi → Hyderabad → Bangalore → Chennai (cost 8).

A13. See code block; heap = next-cheapest node in $O(\log V)$; relaxation = improve dist via neighbour; prev[] = rebuild path backwards.

A14. RIP = distributed BFS (hop metric); OSPF = every router runs Dijkstra on the LSDB; a routing loop is exactly a cycle packets can traverse — detected/avoided via topology knowledge.